CUADERNILLOS DE DESARROLLO DE SOFTWARE

03

PP3 — Coordinación del Proyecto Integrador

El proceso completo de la materia en un solo mapa: sprints, Git Flow, SPEC y ADR, evidencia y defensa.



Práctica Profesionalizante III · 3° año · Ciclo 2026

Tecnicatura Superior en Desarrollo de Software · IES 9-018 · Prof. Paulo Alvarez

Fuente primaria: `github.com/IES9018/proyecto-pp3-2026`

Se lee junto con: `ADI-teoria-y-recursos` · `proyecto-adi-2026`

Índice

Cómo usar este cuadernillo

Tu lugar en la organización

Planificar: backlog, Kanban y objetivos de sprint

Git Flow y la regla dual

SDD en marcha: SPEC, ADR y arnés

Arquitectura viva: C4 en tu repo

Núcleo funcional e interfaz crítica

Calidad local: tests que cuentan

Contratos y seguridad

Mobile medible y despliegue funcional

Documentación legal y licencias

Auditoría, evaluación, bonus y defensa

Cómo usar este cuadernillo

> **Propósito:** que sepas en todo momento qué tenés que hacer, con qué reglas y con qué evidencia. Este cuadernillo no reemplaza el repositorio de coordinación: lo explica, lo relaciona y anticipa lo que viene.

Empezá por entender dónde vive cada cosa, porque en esta materia hay **tres** repositorios + **tu** repositorio individual.

El mapa de los cuatro repositorios

Repositorio	Qué contiene	Lo uso para
`github.com/IES9018/ADI-teoria-y-recursos`	Teoría de Arquitectura y Diseño de Interfaces	Entender los conceptos antes de practicar
`github.com/IES9018/proyecto-adi-2026`	Consignas de TP, plantillas, proyecto modelo A-gobernanza-digital	Copiar plantillas, leer ejemplos reales
`github.com/IES9018/proyecto-pp3-2026`	Reglas de los 3 sprints, checklists, rúbricas, configuraciones	**Este cuadernillo lo explica**, pero el SSOT es el repo
`github.com/IES9018/<tunombre>-<tuproyecto>`	**Tu** proyecto individual	Trabajar, hacer PR, recibir feedback

Repositorio	Contiene	Se conecta con
Teoría (lectura)	U1-U10	→ TP1-6
Consignas (plantillas)	TP1-6, modelo, templates	→ Sprint 1-3
PP3 (reglas)	checklists, rúbricas, config	→ Cierre + defensa
TU REPOSITORIO (tu código)	tu proyecto individual	(todo confluye acá)

Regla: Nunca trabajás directo sobre los repos de la cátedra. Solo sobre el tuyo. Los forks están prohibidos.

> ¿POR QUÉ IMPORTA?
> ¿Por qué importa saber esto? Porque si tenés una duda de "¿cómo era la regla de Git Flow?" no buscás en tu código: buscás en `proyecto-pp3-2026`. Y si te falta una especificación, no la inventás: la copiás de `proyecto-adi-2026/templates/`.

La cadena TP → sprint → cierre

El cronograma real (fuente: `CRONOGRAMA-TPs-2026.md` del repo ADI proyecto) sincroniza los trabajos prácticos de ADI con los sprints de PP3:

TP	Insumo para	Sprint	Período
TP1	SPEC + ADR-001	Sprint 1	24 ago – 18 sep
TP2 / TP3	SPEC v2 + C4 + ADRs + personas y wireframes	Sprint 2	21 sep – 16 oct
TP4 / TP5	OpenAPI + threat model + mobile + presupuestos	Sprint 3 (TP4 se abre 13 oct, entrega 27 oct)	19 oct – 13 nov
TP6	Pipeline CI + release v0.1.0 + postmortem	Cierre (técnico)	entrega 17 nov
—	Defensa integradora (oral)	—	19 nov

Cada unidad de este cuadernillo te dice **en qué sprint y con qué entregable se relaciona**.

Los bloques que vas a encontrar

Toda la colección usa los mismos bloques con el mismo color. El color siempre significa lo mismo.

Bloque	Color	Significa
CONCEPTO CLAVE	Azul	La definición en una frase. Lo más importante.
EN SIMPLE	Verde agua	La idea con palabras de todos: analogía o intuición.
TÉRMINO TÉCNICO	Gris	El nombre profesional en inglés + explicación en español.
¿POR QUÉ IMPORTA?	Ámbar	Qué ganás o qué se rompe si esto no lo sabés.
EJEMPLO REAL	Azul acero	Un archivo concreto del repositorio de la materia.
EJEMPLO DE CÓDIGO	Carbón	Código de ≤ 20 líneas con una sola idea.
PASO A PASO	Violeta	Secuencia numerada 1→n de un procedimiento.
ERROR FRECUENTE	Rojo apagado	El error real + cómo se evita.
NO CONFUNDIR	Rojo/Gris	Dos conceptos que se parecen y no son lo mismo.
RECORDÁ	Amarillo ocre	Regla mnemotécnica o límite que hay que retener.
EN EL PROYECTO	Naranja	Dónde aparece esto en tu TP y en tu sprint, con ruta exacta.
PROBALO	Verde	Tarea concreta, verificable, con entregable mínimo.
¿QUÉ PASARÍA SI...?	Púrpura	Consecuencia hipotética que obliga a razonar.
CHECKPOINT	Fondo gris	2-4 preguntas al cerrar la unidad. Obligatorio.

RECORDÁ

Un color sin significado no existe en esta colección. Si ves un recuadro de color y no entendés qué significa, es un error del cuadernillo: reportalo.

Antes de arrancar

Este cuadernillo es el **número 03** de la colección. Presupone que leíste o tenés a mano los otros dos: - **01** — Arquitectura y Diseño de Interfaces — Teoría (los conceptos) - **02** — Arquitectura y Diseño de Interfaces — Proyecto (las consignas y el modelo)

Pero si no los tenés, no importa: cada unidad es autocontenida y cita la fuente exacta donde profundizar.

> **PROBALO**
>
> Probalo: Abrí `github.com/IES9018/proyecto-pp3-2026` en el navegador. Encontrá las carpetas `sprints/sprint-1/` , `sprints/sprint-2/` y `sprints/sprint-3/` . Anotá qué archivos tiene cada una. Eso es todo el perímetro de lo que necesitás de este repo.

> **CHECKPOINT — RESPONDÉ SIN MIRAR**
>
> 1. ¿Qué cuatro repositorios usás en PP3 y para qué sirve cada uno?
> 2. ¿Qué insumo produce el TP1 y en qué sprint se consume?
> 3. ¿Qué significa que un color “no tiene significado” en esta colección?

Siguiente unidad: *U1 → Tu lugar en la organización*

Tu lugar en la organización

Esta materia tiene un esquema de repositorios distinto al que venís usando: vos no trabajás sobre un repo compartido ni hacés fork de uno existente. **Creás tu propio repositorio dentro de la organización de la cátedra.**

Regla de oro: sin forks, repo propio

Cuando arranca el Sprint 1, no apretás "Fork". Creás un repositorio **nuevo, público, vacío** dentro de `github.com/IES9018/` con el nombre:

```
IES9018/<tu_nombre>-<nombre_de_tu_proyecto>
```

Ejemplo: `IES9018/raul-crm`, `IES9018/maria-sistema-licencias`.

EN SIMPLE

no es "me copio el repo de la cátedra y lo modifico". Es "creo el mío desde cero y uso los de la cátedra como referencia". Si hicieras fork, tus PR intentarían ir al repo original (el del docente), y eso rompe todo el esquema de auditoría.

CONCEPTO CLAVE

Organización IES9018: el espacio de GitHub donde están todos los repositorios de la carrera. Tu repo individual debe estar acá, no en tu cuenta personal. Si lo creás en tu cuenta, el bot de seguimiento no puede auditorlo.

Cómo se crea tu repositorio

1. Entrás a `github.com/IES9018` (estás invitado como miembro de la organización).
2. New repository → nombre: `<tunombre>-<tuproyecto>` → público → **sin inicializar** (sin README, sin .gitignore, sin licencia).
3. En tu terminal local:

```
git init
git add .
git commit -m "chore: initial commit"
git remote add origin git@github.com:IES9018/<tunombre>-<tuproyecto>.git
git push -u origin main
```

4. Subí los archivos obligatorios del Sprint 1 copiados desde las plantillas.

ERROR FRECUENTE

crear el repo como privado. No puede: el bot de seguimiento y el docente necesitan acceso de lectura pública. Si lo necesitás privado por política de datos (proyecto con información sensible), hablalo con el docente **antes** de crear el repo.

El seguimiento: sos auditor todos los días

La cátedra tiene un **script determinista** (sin IA) que corre todos los días a las 10:00 Argentina. Revisa tu repositorio y anota automáticamente:

- ¿Tenés SPEC.md en la raíz? Sí/No.
- ¿Tenés `docs/adr/` con al menos un ADR? Sí/No.
- ¿Tenés `.opencoderules`? Sí/No.
- ¿Hay PRs abiertos? ¿Están mergeados?
- ¿Los commits siguen el formato convencional?

> **¿POR QUÉ IMPORTA?**
> Porque si algo falta, recibís un **Issue automático** en tu repo con los pasos a seguir. No es un castigo: es el mecanismo para que no te atrases. Si ignorás los issues, el docente lo ve reflejado en el seguimiento.

El registro de seguimiento es interno del docente. Vos no ves el tablero general, pero ves tu propio historial en los Issues de tu repositorio.

> **RECORDÁ**
> **El bot no interpreta, no opina, no perdona:** si el archivo no está, no está. No importa si "está por subirse". Subilo con un commit y pushealo; si el bot corre antes, te llega el Issue igual.

Roles de auditoría

El docente audita con dos sombreros:

Rol	Qué mira	Evidencia primaria
Capataz	Verificación de cumplimiento: SPEC completa, checklist marcados, Non-Goals explícitos	PRs mergeados, informes de auditoría
Arquitecto de Obra	Decisión técnica: ADR justificado, diagramas C4 trazables, contratos coherentes	`SPEC.md`, `docs/adr/`, `docs/arquitectura/`

Tu trabajo no es "adivinar qué quiere el docente": la rúbrica es determinista y pública (`docs/auditoria/RUBRICA_AUDITORIA_S1.md` en el repo PP3). Si cumplís los puntos, aprobás.

Anatomía de tu repositorio

Al final del Sprint 1, tu repo debería verse así:

```
SPEC.md                      ← especificación del proyecto
.opencoderules               ← reglas del agente de IA (arnés)
INSTRUCTIONS.md (opcional)   ← complemento del arnés
docs/
  adr/
    ADR-001-stack-tecnologico.md
  arquitectura/              ← a partir del Sprint 2
  pruebas/                   ← a partir del Sprint 2
  auditoria/
    auditoria-sprint1.md
    auditoria-sprint2.md
    auditoria-sprint3.md
  seguridad/                 ← a partir del Sprint 3
  deploy/                    ← a partir del Sprint 3
```

EN EL PROYECTO

En tu sprint (Sprint 1, Fase 0-1): creá la estructura de carpetas vacía (`docs/adr/`, `docs/auditoria/`) aunque todavía no tengan archivos. El bot busca la carpeta, no solo el archivo.

Cómo fluye el feedback

```
1. Hacés commit + PR en tu repo (con plantilla)
2. El bot corre a las 10:00 y registra el estado
3. Si falta algo → Issue automático en tu repo
4. Corregís → nuevo commit → nuevo PR
5. El docente revisa PRs en lote y deja comentarios
6. Cuando está todo → mergeás vos mismo
```

¿QUÉ PASARÍA SI…?

Ignoraras un Issue del bot durante una semana? Acumulás deuda técnica de gestión: el docente ve que no corregiste, el Sprint 2 arranca con la SPEC desactualizada y el arrastre se nota en la defensa. Cada sprint se cierra con su informe de auditoría.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Por qué se crea un repo propio y no un fork? ¿Dónde debe vivir ese repo?
2. ¿Qué revisa el bot de seguimiento a las 10:00 y qué recibís si falta un archivo?
3. ¿Qué evidencia primaria mira el docente como Capataz y cuál como Arquitecto de Obra?

Siguiente unidad: *U2 → Planificar: backlog, Kanban y objetivos de sprint*

Planificar: backlog, Kanban y objetivos de sprint

En PP3 no trabajás con "lo que tocó hoy" ni con "lo que falta para terminar el TP". Trabajás con **sprints** que tienen un **objetivo verificable** y un **backlog** priorizado.

Scrum y Kanban: lo mínimo que necesitás

La cátedra usa una mezcla práctica: el **backlog** y los **objetivos de sprint** vienen de Scrum; la **visualización del avance** y los **límites de trabajo en progreso** (WIP) vienen de Kanban.

> CONCEPTO CLAVE
>
> **Backlog:** lista priorizada de todo lo que el proyecto necesita. No es una wishlist: cada ítem tiene un criterio de aceptación verificable. El backlog vive en el **tablero Kanban** de GitHub Projects.

> CONCEPTO CLAVE
>
> **Objetivo de sprint:** una frase que describe qué va a estar funcionando al final del sprint. No es "hacer cosas". Es "el usuario puede registrarse y ver su perfil". Medible, demostrable.

> CONCEPTO CLAVE
>
> **Tablero Kanban:** el panel visual con columnas *Por hacer / En progreso / Hecho*. Cada tarea es una tarjeta que se mueve de izquierda a derecha. El tablero oficial de la materia está en GitHub Projects y tiene un tutorial paso a paso en `proyecto-pp3-2026/docs/TUTORIAL-kanban.md`.

Cómo se construye el objetivo de sprint

No es "terminar el TP2". El TP2 es un insumo. El objetivo se escribe así:

Elemento	Pregunta que responde	Ejemplo real (Sprint 2)
Verbo	¿Qué se logra?	"Implementar y documentar"
Feature	¿Qué parte del sistema?	"el núcleo funcional con SPEC v2 actualizada"
Evidencia	¿Cómo se comprueba?	"≥ 10 tests verdes corriendo localmente"
Criterio de cierre	¿Quién dice que está?	"PR final mergeado con todos los checklists completos"

El objetivo del Sprint 2 se lee así:

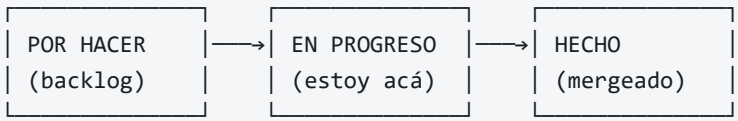
"Implementar y documentar el núcleo funcional con SPEC v2 actualizada, ≥ 10 tests verdes locales, wireframes de 2 pantallas críticas, y PR final mergeado con todos los checklists completos."

> **EN SIMPLE**
> no es “entregar algo”, es “que algo funcione y se pueda verificar”. Si tenés 200 líneas de código pero 0 tests, no cumpliste el objetivo. Si tenés 0 líneas nuevas pero la SPEC v2 está completa y los tests están verdes, avanzaste.

El tablero Kanban paso a paso

El repo PP3 tiene un tutorial dedicado (`docs/TUTORIAL-kanban.md`) con captura de pantalla. En resumen:

1. Entrá a GitHub → Projects → “PP3 2026 — Seguimiento”.
2. Creá una tarjeta por cada entregable del sprint (los `DEL-Sx-xx`).
3. Mové la tarjeta a *En progreso* cuando arrancás.
4. Cuando el PR está mergeado → mové a *Hecho*.
5. Al cerrar el sprint, todas las tarjetas deberían estar en *Hecho*.



> **ERROR FRECUENTE**
> poner todo el TP como una sola tarjeta “Sprint 2”. Un sprint no es una tarea: es un conjunto de tareas que entregan un objetivo. Si tenés una sola tarjeta, no podés visualizar qué está bloqueado y qué avanzó.

Registro de avance y bloqueos

Cuando trabajes, actualizá dos cosas:

Cuándo	Qué registrar	Dónde
Arrancás una tarea	Mové la tarjeta a *En progreso* + comentá quién arranca	Tablero Kanban
Te trabás	Comentá el bloqueo en la tarjeta: “me falta definir X para seguir”	Tablero Kanban
Terminás	Mergeá el PR + mové la tarjeta a *Hecho*	GitHub
Cierre de sprint	Completá el informe de auditoría	`docs/auditoria/` en TU repo

¿POR QUÉ IMPORTA?

Porque el docente no te pregunta “¿cómo vas?” todos los días. Revisa el tablero y los Issues. Si tu tablero está vacío o tu tarjeta lleva 5 días en

En progreso

sin comentarios, se nota.

Los 3 sprints del año

Sprint	Fechas	Objetivo resumido	Entregables clave
Sprint 1	24 ago — 18 sep	Setup + SPEC + ADR-001 + arnés + Git Flow	`SPEC.md`, `ADR-001`, `auditoria-sprint1.md`
Sprint 2	21 sep — 16 oct	Núcleo funcional + C4 + tests + wireframes	`SPEC v2`, C4, `reporte-sprint2.md`, `auditoria-sprint2.md`
Sprint 3	19 oct — 13 nov	API con OpenAPI + seguridad + mobile + despliegue	contrato YAML, threat model, presupuestos, `deploy.md`
Cierre (técnico)	17 nov	Pipeline CI + release v0.1.0 con CHANGELOG + informe final/demo	CI verde + release + `docs/informe-final.md`
Defensa integradora (oral)	19 nov	—	defensa individual

Las fechas exactas están en `sprint-pp3-config.json` y en los `sprint-config.json` de cada carpeta. Este cuadernillo no las duplica: te dice dónde mirar.

RECORDÁ

El cronograma no es tentativo: cada sprint tiene una semana de cierre con fechas límite para mergear PRs. Pasada esa fecha, los checklists se cierran y el sprint se audita con lo que hay.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué distingue un objetivo de sprint de una lista de tareas? ¿Qué 4 elementos lo componen?
2. ¿Qué columnas tiene el tablero Kanban y cuándo movés una tarjeta a *Hecho*?
3. ¿Qué registrás en la tarjeta cuando te trabás con una tarea?

Siguiente unidad: *U3 → Git Flow y la regla dual*

Git Flow y la regla dual

El flujo de trabajo con Git es **obligatorio y auditado**. No es una sugerencia: si no hay ramas, commits con formato y PRs con plantilla, no hay evidencia que evaluar.

El flujo

Todo cambio arranca igual:

```
Issue en Kanban → rama feature/<tema> → commits convencionales → PR con plantilla → checklists completos → merge
```

CONCEPTO CLAVE

Rama `feature/<tema>` **:** cada cambio tiene su propia rama. No se trabaja sobre `main` ni sobre `develop` . Si estás arreglando el login, la rama se llama `feature/login` . Si estás agregando tests, `feature/tests-integracion` .

PASO A PASO

1. Partiendo de la rama base de tu repo (si ya tenés `develop` : `git checkout develop && git pull` ; en el Sprint 1, si tu repo recién creado solo tiene `main` , partí de `main`)
2. Creás la rama: `git checkout -b feature/<tema>`
3. Hacés commits con formato convencional.
4. Pusheás: `git push -u origin feature/<tema>`
5. Abrís el Pull Request usando la plantilla oficial.
6. Completás la plantilla del PR: trazabilidad del Issue + Quality & Security Gate + Git Flow & Convenciones.
7. Cuando está aprobado → **mergeás vos mismo** al destino correspondiente (en tu repo del Sprint 1: `main` ; en el flujo completo: `develop`).
8. Al final del sprint: PR de `develop` → `main` .

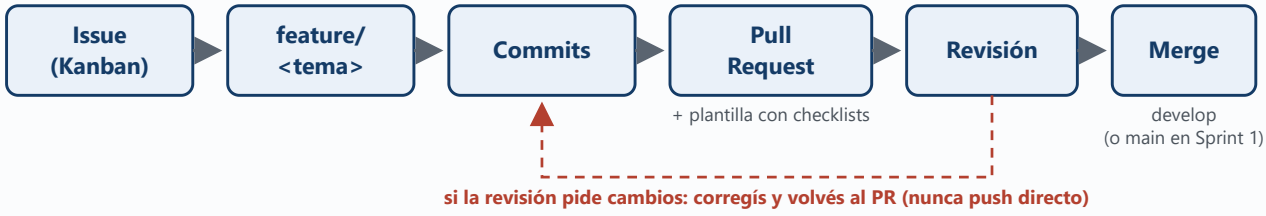


Figura — ¿Cómo viaja un cambio desde una tarea hasta la versión estable? Representa el flujo de `docs/arquitectura/DIAGRAMAS_REFERENCIA.md` *del repo PP3.*

La referencia visual completa está en `docs/arquitectura/DIAGRAMAS_REFERENCIA.md` del repo PP3, con dos diagramas Mermaid: el flujo de trabajo y el ciclo de vida de un PR.

La regla dual (esto es importante)

No hay una sola forma de hacer PR: hay dos según **dónde** estés trabajando.

Escenario	Tu repo individual (`IES9018/<tunombre>-<tuproyecto>`)	Repo de coordinación (`IES9018/proyecto-pp3-2026`)
Quién mergea	**Vos mismo** después de completar los checklists	El docente (o vos con su aprobación)
Rama protegida	`main` y `develop` — no se puede pushear directo	`main` protegida: PR obligatorio con **1 aprobación**
Historial	Reversible: si el merge rompe algo, podés revertir el merge	Estable: no se revierte sin aviso
Cuándo se mergea	Cuando los checklists del PR están completos y los tests pasan	Cuando el docente aprueba la revisión

EN SIMPLE

en tu repo, vos sos el dueño — mergeás cuando está listo y si se rompe, lo revertís. En el repo de coordinación, el dueño es la cátedra — necesitás que alguien más apruebe.

ERROR FRECUENTE

creer que la regla dual significa “en mi repo puedo pushear directo a main”. **No.** El push directo a `main` y a `develop` está bloqueado por GitHub en ambos repos. Nadie —ni siquiera el docente— puede pushear directo. Siempre es por rama + PR.

Commits convencionales

Cada commit tiene un prefijo que dice **qué tipo de cambio** es:

Prefijo	Cuándo usarlo
`feat:`	Una funcionalidad nueva (ej: `feat: agrega endpoint de login`)
`fix:`	Corrección de un error (ej: `fix: valida email vacío en registro`)
`refactor:`	Cambio de código sin cambiar comportamiento (plantilla oficial)
`docs:`	Cambios en documentación (ej: `docs: actualiza README con setup`)
`ci:`	Cambios en pipeline o configuración de CI (ej: `ci: agrega lint al workflow`)
`chore:`	Mantenimiento o dependencias (lo lista el README de la cátedra)

El formato exacto: `tipo: mensaje en presente, sin mayúscula inicial, sin punto final`.

```
feat: agrega endpoint POST /solicitudes
fix: valida formato de email en registro
docs: agrega sección de Non-Goals a SPEC-template
```

> RECORDÁ
>
> **Un commit no es "avances del día".** Es la unidad más chica de cambio con sentido. Si tocaste dos features distintos, son dos commits. Si arreglaste un bug y documentaste, son dos commits.

La plantilla del PR

Cuando abrís un Pull Request, GitHub te muestra automáticamente la plantilla institucional (archivo: `.github/PULL_REQUEST_TEMPLATE.md` del repo PP3).

La plantilla oficial (`.github/PULL_REQUEST_TEMPLATE.md` del repo PP3) tiene **tres zonas** que completás antes de mergear. Visualmente se muestra como documento/formulario:

Tipo de Cambio

- feat / fix / refactor / ci / docs

Trazabilidad de Issue

Closes #

Checklists de Supervisión Crítica

Quality & Security Gate

- No se exponen credenciales, tokens ni secrets

- Entradas validadas contra inyección
- Código asistido por IA auditado línea por línea
- Pruebas locales exitosas

Git Flow & Convenciones

- Commits con convención (feat:, fix:, ci:, docs:)
- Rama con nomenclatura feature/, bugfix/ o hotfix/
- Sin conflictos de merge con develop

> CONCEPTO CLAVE
>
> **Checklist:** lista de verificación que se tilda antes de considerar algo terminado. No es un trámite: si marcás algo sin haberlo verificado, la auditoría lo detecta porque tu PR queda como evidencia.

Ciclo de vida de un PR

El estado de cada PR se ve en el diagrama oficial del repo:

```
Draft → In Review → (Changes Requested → In Review) → Approved → Merged
```

No es lineal: si el revisor pide cambios, el PR vuelve a

In Review

hasta que esté correcto.

> ¿QUÉ PASARÍA SI...?
>
> Mergearas un PR con un checklist sin marcar? Queda en el historial. La auditoría revisa PRs mergeados, no solo abiertos. El checklist sin marcar es evidencia de que no se verificó algo.

Convenciones adicionales

Regla	Detalle
Commits en español	El mensaje va en español, el código en inglés (nombres de variables, funciones).
Sin mayúscula inicial	`feat: agrega login` y no `feat: Agrega login`.
Sin punto final	El mensaje no lleva punto.
Un commit por cambio	Si necesitás "y además", partí en dos commits.

Repaso rápido

FLUJO COMPLETO (cuando tu repositorio ya trabaja con `develop`):

```
1. feature/<tema> desde develop
2. commits con prefijo convencional
3. PR con la plantilla oficial completa
4. mergeás vos mismo a develop (en TU repo)
5. al final del sprint: develop → main
```

Excepción documentada (Sprint 1): si tu repo recién creado solo posee `main`, el PR inicial va a `main`, según el CHECKLIST oficial del Sprint 1. No hay una tercera modalidad.

EN EL PROYECTO

En tu sprint (Sprint 1, Fase 3): después de crear la SPEC y el ADR, abrí tu primer PR hacia `main` de TU repo — así lo pide el checklist oficial del Sprint 1. En tu repo recién creado todavía no existe `develop`; cuando la adoptes (flujo completo), el destino de tus PR pasa a ser `develop`. En ambos casos: nunca push directo, siempre rama + PR con la plantilla, y mergeás vos con los checklists completos.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué prefijo lleva un commit que corrige un bug? ¿Y uno que agrega un endpoint?
2. Explicá la regla dual: ¿quién mergea en tu repo y quién en el repo de coordinación?
3. ¿Cuáles son las secciones principales de la plantilla del PR y qué verifican los dos checklists de supervisión crítica?

Siguiente unidad: *U4 → SDD en marcha*

SDD en marcha: SPEC, ADR y arnés

El acrónimo SDD (Spec-Driven Development) suena a teoría, pero en PP3 es el **primer entregable del Sprint 1**. No es "leer sobre SDD", es escribir los archivos que lo implementan.

El archivo que manda: SPEC.md

Al arrancar el Sprint 1, copiás la plantilla institucional desde `proyecto-adi-2026/templates/SPEC-template.md` a la raíz de tu repo y la completás.

La SPEC tiene 6 secciones, y cada una responde una pregunta distinta:

Sección	Pregunta	Qué incluye
1. Contexto y Propósito	¿Qué problema resuelve el sistema?	Una frase de contexto + una de propósito.
2. Requerimientos Funcionales	¿Qué hace exactamente?	Lista de `RF-01`, `RF-02`... con criterios de aceptación.
3. Non-Goals	¿Qué NO hace y por qué?	Exclusiones explícitas con justificación técnica.
4. Stack y Restricciones	¿Con qué se construye?	Tecnologías, versiones, ADR citados.
5. Contratos de datos	¿Cómo se comunican las partes?	Endpoints, schemas, formatos.
6. Criterios de aceptación	¿Cómo sé que está terminado?	Condiciones verificables por test.

> CONCEPTO CLAVE
>
> **Non-Goal:** un objetivo que se decide explícitamente **no** perseguir en esta versión. No es una omisión por descuido: es una decisión documentada. Ejemplo: "No es una app móvil nativa (ver ADR-006)". Si después alguien pregunta "¿por qué no hay app?", la respuesta está en la SPEC.

> EN SIMPLE
>
> la SPEC es el contrato entre vos y tu proyecto. Lo que no está en la SPEC, no existe. Lo que está en Non-Goals, no se puede reclamar. Lo que está en RF, se tiene que poder mostrar funcionando.

De v1 a v2

La SPEC no es estática. Al abrir el Sprint 2, se actualiza a **v2** con:

- Una nueva sección "Restricciones arquitectónicas" que cita tus ADRs.
- Un changelog al pie que registra qué cambió entre v1 y v2.

```
SPEC.md v1 (Sprint 1): alcance inicial, stack, Non-Goals
       ↓
SPEC.md v2 (Sprint 2): + restricciones arquitectónicas citando ADRs
       ↓
v3, v4… la SPEC sigue versionándose: el TP5 pide la v5
con los RNF medibles como requisitos
```

ERROR FRECUENTE

no tocar la SPEC después del Sprint 1. Una SPEC congelada es peor que no tener SPEC: miente sobre el estado real del proyecto. Si el código cambió pero la SPEC no, la trazabilidad se rompe y la auditoría lo detecta.

Las decisiones que se escriben: los ADR

Cada decisión técnica importante se documenta como un **Architecture Decision Record (ADR)**. La plantilla está en `proyecto-adi-2026/templates/ADR-template.md`.

Estructura de un ADR:

```
# ADR-001: Stack Tecnológico

## Contexto
¿Qué problema estamos resolviendo y por qué no se resuelve solo?

## Decisión
Qué elegimos y en qué consiste.

## Alternativas Consideradas
Al menos 2 alternativas REALES que evaluamos y descartamos, con el motivo.

## Consecuencias
Qué cambia a partir de esta decisión. Qué se gana y qué se pierde.
```

En el Sprint 1, el ADR obligatorio es el **ADR-001** que define el stack tecnológico. Debe contener **al menos 2 alternativas descartadas** con su justificación.

EN EL PROYECTO

En tu sprint (Sprint 1, Fase 1): creá `docs/adr/ADR-001-stack-tecnologico.md` con:

- Contexto: qué sistema vas a construir y con qué propósito.
- Decisión: stack elegido (lenguaje, framework, base de datos, hosting).
- Alternativas: al menos 2 (ej: FastAPI vs Flask, React vs Vue, SQLite vs PostgreSQL).
- Consecuencias: qué facilitás y qué dificultás con esta decisión.

El proyecto modelo A-gobernanza-digital tiene 7 ADR (ADR-001 a ADR-007) que podés leer como ejemplo en `proyecto-adi-2026/03_ARQUITECTURA/adr/`.

El arnés de IA: .opencoderules

El archivo `.opencoderules` en la raíz de tu repo limita el comportamiento de OpenCode (o del agente que uses). Define:

- **Alcance permitido:** qué archivos y carpetas puede tocar el agente.
- **Estándares obligatorios:** formato de commits, estilo de código, plantillas a usar.
- **Prácticas prohibidas:** qué no puede hacer el agente (ej: modificar SPEC sin avisar).

```
🛡️ ALCANCE PERMITIDO
  - src/ → código fuente
  - docs/ → documentación y ADR
  - SPEC.md → leer, sugerir cambios en PR

📋 ESTÁNDARES OBLIGATORIOS
  - Commits: feat/fix/docs/ci/chore
  - Tests: pytest con aserciones significativas
  - Contratos: OpenAPI 3.x primero

🚫 PRÁCTICAS PROHIBIDAS
  - No modificar SPEC.md sin PR
  - No incluir secrets en ningún archivo
  - No pushear directo a main o develop
```

¿POR QUÉ IMPORTA?

Porque el arnés es lo que permite que uses IA sin que la IA decida por vos. El docente revisa que el arnés esté configurado correctamente (que no sea el default vacío). Si no hay arnés, el código asistido no tiene reglas → el PR no se puede auditar.

El archivo `INSTRUCTIONS.md` es opcional y complementa el arnés con instrucciones en lenguaje natural para el agente.

PASO A PASO

Puesta en marcha del SDD 1. Copiá `SPEC-template.md` a la raíz → completá secciones 1 a 4 (Sprint 1). 2. Creá `docs/adr/ADR-001.md` con stack y 2 alternativas descartadas. 3. Creá `.opencoderules` con alcance, estándares y prohibiciones. 4. Commit + PR con la plantilla. 5. El docente revisa estructura (no contenido): si la plantilla está completa, Fase 1 aprobada.

RECORDÁ

La SPEC se escribe antes del código. Si ya tenés código sin SPEC, detenete, escribí la SPEC, después seguí. El orden no es burocrático: es la diferencia entre un proyecto con rumbo y un proyecto que "ya veremos".

> **CHECKPOINT — RESPONDÉ SIN MIRAR**
>
> 1. ¿Qué secciones tiene la SPEC y qué pregunta responde cada una?
> 2. ¿Qué debe contener como mínimo el ADR-001?
> 3. ¿Qué tres zonas define el arnés `.opencoderules` ?

Siguiente unidad: *U5 → Arquitectura viva: C4*

Arquitectura viva: C4 en tu repo

El Sprint 2 arranca con una tarea que no es código: dibujar la arquitectura de tu proyecto en **2 niveles del modelo C4** (contexto y contenedores). No es un diagrama decorativo: es el mapa contra el que se mide cada decisión técnica.

Qué es C4

C4 es una notación de diagramas arquitectónicos que va de lo más general a lo más detallado en 4 niveles. En PP3 se piden los primeros 2:

```
C1 — Contexto        | el sistema y sus actores externos
C2 — Contenedores    | las piezas grandes (frontend, API, DB)
```

CONCEPTO CLAVE

C1 (Contexto): muestra tu sistema como una caja en el centro y las entidades con las que se comunica (usuarios, APIs externas, sistemas legacy) como otras cajas alrededor. No importan los detalles internos: importa qué límites tiene tu sistema.

CONCEPTO CLAVE

C2 (Contenedores): desglosa la caja central de C1 en las piezas que la componen: un frontend web, una API, una base de datos, un worker de tareas, etc. Cada contenedor se conecta con flechas que indican el protocolo de comunicación (HTTP, WebSocket, SQL).

EN SIMPLE

C1 es la foto satelital (se ve el barrio). C2 es el zoom a la manzana (se ven los edificios). El nivel C3 (componentes) y C4 (código) no se piden en PP3, pero si los dibujás, mejor.

Dónde va y qué debe contener

Los diagramas se guardan en `docs/arquitectura/` con el formato que prefieras (Mermaid, PNG, SVG). Lo importante no es la herramienta: es que **cada elemento del diagrama sea trazable a un elemento de la SPEC o de un ADR**.



Figura — ¿Cómo se mantiene la trazabilidad entre lo que decidí, diseñé, implementé y verifiqué? Es la regla de oro de la clase SDD-C4-SPEC del repo ADI proyecto.

Regla de oro del C4 en PP3

> **Si aparece en el diagrama, tiene que aparecer en la SPEC o en un ADR. Y viceversa.**

Ejemplo: si en tu C2 dibujás “Base de Datos”, en la SPEC o en un ADR tiene que estar la decisión de qué base de datos es y por qué. Si en la SPEC decís “PostgreSQL”, en C2 no podés dibujar “SQLite”.

> ERROR FRECUENTE
>
> dibujar un C2 con 8 contenedores cuando la SPEC describe 3 features. Señal de sobreingeniería: estás pensando en infraestructura antes de tener problemas de infraestructura. Si tenés dudas, arrancá con 2 contenedores: frontend y API (con DB dentro de API como módulo).

Ejemplo real del proyecto modelo

El repo `proyecto-adi-2026` tiene el C4 completo de A-gobernanza-digital en `03-proyectos/A-gobernanza-digital/03_ARQUITECTURA/`. Podés leerlos como referencia:

Archivo	Nivel	Contenido
`diagrama-contexto-c4.md`	C1	Sistema Gobernanza Digital y sus actores: Solicitante, Admin Técnico, Directivo, Consejo Directivo, Comunidad Educativa
`diagrama-contenedores-c4.md`	C2	Servidor escolar Debian 12: Nginx → React + Vite (SPA) → API, con protocolos HTTPS y SQL
`diagrama-componentes-c4.md`	C3	Interior de la API por componentes

> EN EL PROYECTO
>
> En tu sprint (Sprint 2, Fase 1): actualizá tu `SPEC.md` a v2 agregando la sección “Restricciones arquitectónicas” y creá `docs/arquitectura/` con los diagramas C1 y C2. No necesitan ser perfectos: necesitan ser trazables.

Herramientas para dibujar C4

Herramienta	Ventaja	Dónde obtenerla
Mermaid (diagramas C4)	Se escribe como texto, vive en el repo, el pipeline lo renderiza	Sintaxis: `c4context` / `c4container`
Draw.io / diagrams.net	Interfaz visual, exporta a SVG	Web o app de escritorio
Lápiz y papel → foto → repo	Válido para C1 si no usás otra	La foto va en `docs/arquitectura/`

La cátedra prefiere Mermaid porque el diagrama se versiona como texto y no se pierde. El repo PP3 tiene ejemplos en `docs/arquitectura/DIAGRAMAS_REFERENCIA.md`.

> **RECORDÁ**
>
> **Un diagrama sin etiqueta de pregunta no sirve.** Cada diagrama de este cuadernillo responde una pregunta. El tuyo también: al pie del diagrama, escribí "¿Qué pregunta responde este diagrama?" si no podés responderla, el diagrama no está cumpliendo su función.

> **CHECKPOINT — RESPONDÉ SIN MIRAR**
>
> 1. ¿Qué muestra C1 (contexto) y qué muestra C2 (contenedores)?
> 2. ¿Cuál es la regla de oro entre los diagramas y la SPEC/los ADR?
> 3. ¿Por qué la cátedra prefiere Mermaid sobre una imagen exportada?

Siguiente unidad: *U6 → Núcleo funcional e interfaz crítica*

Núcleo funcional e interfaz crítica

El Sprint 2 tiene dos entregas grandes que van de la mano: los features del sistema tienen que funcionar **en local** y las pantallas más importantes tienen que estar **diseñadas**.

Qué es "núcleo funcional"

No es todo el sistema. Son los features que definen el propósito de tu proyecto: si no funcionan, el sistema no sirve para nada.

> CONCEPTO CLAVE
>
> **RF-01..RF-n operativos:** los Requerimientos Funcionales principales de tu SPEC corriendo en tu máquina local. No en un servidor, no en producción: en local, con tus herramientas, verificables. Si la SPEC dice "RF-01: el usuario puede registrarse" y ves el formulario de registro funcionando, está operativo.

La regla para elegir cuáles implementar:

El feature...	Incluilo si...	Dejalo para después si...
RF-01 a RF-n (los primeros)	Define el propósito del sistema	Es una mejora estética
Tiene validación de datos	Muestra que manejás errores	Depende de un servicio externo que no configuraste
Lo usa el flujo principal	Sin esto no hay demo	Solo lo usa el admin una vez por mes

> ERROR FRECUENTE
>
> implementar 10 features a medias en vez de 3 features completos. Un feature completo se puede demostrar, tiene test y está documentado. Un feature a medias no sirve ni para mostrar ni para auditar.

Personas y journeys: antes del diseño

Antes de dibujar cualquier pantalla, definí **quién la usa** y **qué necesita hacer**.

> CONCEPTO CLAVE
>
> **Persona:** perfil ficticio de un usuario típico con nombre, objetivos, habilidades técnicas y contexto. No es un estereotipo: es una herramienta de diseño. "María, 45 años, técnico municipal, usa la compu 3 horas por día, prefiere formularios simples."

CONCEPTO CLAVE

Journey map: el recorrido que hace una persona para completar una tarea, paso a paso, con sus emociones, dudas y momentos de fricción. "1. Abre el sistema → 2. Busca el botón de carga → 3. No lo encuentra → 4. Pregunta a un compañero → 5. Completa el formulario."

Las 2 pantallas críticas

Del journey map, sacás las pantallas que tienen más impacto en el éxito de la tarea. Esas son las que **wireframeás**.

EN SIMPLE

no wireframeés las 20 pantallas del sistema. Wireframeá las 2 que, si están mal, hacen que el usuario abandone. Para el resto, alcanza con describirlas en texto en la SPEC.

Wireframe (baja fidelidad): dibujo esquemático sin colores, sin tipografía final, sin imágenes. Solo estructura. Se puede hacer en papel, en Figma, en Mermaid o en Markdown.

El repositorio de ADI proyecto tiene 4 wireframes reales en `06_INTERFAZ_USUARIO/wireframes/` que podés usar como referencia de formato.

Accesibilidad básica

No se trata de cumplir todas las pautas WCAG. Se trata de no cometer los errores que hacen que una interfaz sea inusable para alguien:

Regla mínima	Cómo verificarlo
Contraste suficiente	Texto sobre fondo: probá en blanco y negro. Si no se lee, no pasa.
Tamaño mínimo de blancos táctiles	Botones y enlaces: ≥ 48 × 48 px (objetivo táctil, no visual).
Textos en imágenes	No. Si es información, va como texto HTML.
Etiquetas visibles	Cada campo de formulario tiene una etiqueta visible, no solo placeholder.

¿POR QUÉ IMPORTA?

¿Por qué importa la accesibilidad? Porque no es un "plus". Los criterios de evaluación incluyen accesibilidad en el 40% de implementación. Si la auditoría detecta que un botón tiene menos de 48 px de target táctil, es una observación documentada.

EN EL PROYECTO

En tu sprint (Sprint 2, Fase 2-3): después de tener los RF operativos, definí 2 personas, dibujá su journey map, wireframeá las 2 pantallas críticas e implementalas respetando accesibilidad básica. Cada pantalla va a su PR con su propio feature branch.

RECORDÁ

“Operativo en local” ≠ “desplegado”. No necesitás un servidor público para el Sprint 2. Necesitás que funcione en tu máquina y que se pueda demostrar. El deploy llega en el Sprint 3.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué significa que un RF está “operativo en local”? ¿En qué sprint se exige?
2. ¿Por qué se wireframean solo 2 pantallas críticas y no todas?
3. Nombrá dos reglas mínimas de accesibilidad y cómo se verifican.

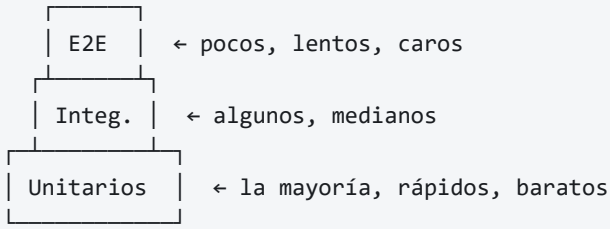
Siguiente unidad: *U7 → Calidad local: tests que cuentan*

Calidad local: tests que cuentan

El Sprint 2 exige ≥ **10 tests verdes corriendo localmente**. No es un número al azar: son los tests de tu núcleo funcional.

Qué tipo de tests

La pirámide de testing aplicada a tu proyecto:



Para el Sprint 2, la prioridad son **tests unitarios**: rápidos, sin red, sin base de datos real (mockeada o en memoria). Si te sobra tiempo, agregá 1 o 2 de integración.

> CONCEPTO CLAVE
>
> **Test unitario:** prueba que verifica una función o método de forma aislada, sin depender de otros componentes. Si tu función `validar_email()` recibe un string y devuelve `True/False`, el test unitario prueba ambos casos.

Non-Goals de testing

Tan importante como lo que testés es saber **qué no estás testeando**. Se documenta en el reporte del sprint:

```
Cubierto:
  - validación de datos de entrada (unitario)
  - lógica de negocio del feature principal (unitario)
  - flujo de registro http 201 (integración)

NO cubierto (Non-Goals de testing):
  - rendimiento bajo carga → se define en Sprint 3
  - seguridad → se cubre con threat model en Sprint 3
  - E2E en navegador → no hay playwright configurado aún
```

> EN SIMPLE
>
> un test que no dice qué NO cubre es un test que promete más de lo que puede. El Non-Goal de testing es tan importante como el de la SPEC: protege tu alcance.

El reporte del sprint

Cada sprint se cierra con un informe de auditoría en `docs/auditoria/auditoria-sprint2.md`. Debe contener:

- **Lista de tests** con lo que cubre cada uno (referenciando RF si aplica).
- **Non-Goals de testing** (explícitos, tabla).
- **Errores hallados en código asistido por IA** y sus correcciones con links a PRs.

El formato es libre, pero la evidencia es obligatoria. Si no hay reporte, el sprint no está cerrado.

```
auditoria-sprint2.md
├── Tests (≥ 10): RF-01..RF-n cubiertos
├── Non-Goals de testing
├── Errores IA detectados → correcciones en PR-xx, PR-xy
└── Estado del sprint: PASS / FAIL
```

EN EL PROYECTO

En tu sprint (Sprint 2, Fase 4): escribí la suite de tests, corrélos, verificá que estén verdes. Después creá `docs/pruebas/reporte-sprint2.md` con el detalle. Al cerrar el sprint, el informe de auditoría va a `docs/auditoria/auditoria-sprint2.md`.

Bonus: apéndice de testing de consistencia

El apéndice avanzado de Testing de Consistencia y Trazabilidad (bonus de **+1 punto**) se explica completo en la U11. En esta unidad solo lo mencionamos porque se construye sobre los tests del Sprint 2:

Un flujo crítico con 3 escenarios (happy path, validación, control de acceso) + matriz de trazabilidad + datos sintéticos. Si te interesa, leé la U11 antes de cerrar el Sprint 2.

RECORDÁ

Una prueba sin aserciones significativas no cuenta como evidencia válida de testing. Lo establece así el README de la cátedra para el bonus de testing de consistencia — y es la regla profesional en general: un test que no verifica nada, no verifica.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué nivel de la pirámide prioriza el Sprint 2 y por qué?
2. ¿Qué es un Non-Goal de testing y dónde se documenta?
3. ¿Qué hace que una prueba no cuente como evidencia válida de testing?

Siguiente unidad: *U8 → Contratos y seguridad*

Contratos y seguridad

En el Sprint 3 primero se escribe el contrato, después se implementa la API, después se modelan las amenazas y después se aplican mitigaciones. Ese orden no es casual.

Contratos antes que código: OpenAPI

El entregable `DEL-S3-01` pide un archivo `api-contracts.yaml` (OpenAPI 3.x) con **al menos 5 endpoints críticos** de tu sistema, validados por un linter sin errores.

> CONCEPTO CLAVE
>
> **OpenAPI 3.x:** estándar para describir APIs REST. En un archivo YAML o JSON definís: rutas, métodos, parámetros, schemas de request/response, códigos de error y esquemas de seguridad. Es el contrato entre frontend y backend.

La regla: **todo cambio de endpoint empieza en el YAML.** Si necesitás un nuevo campo en la respuesta, primero lo escribís en `api-contracts.yaml`, después lo implementás en el código.

```
api-contracts.yaml (contrato)
       |
       ▼
Implementación (FastAPI / Express / etc.)
       |
       ▼
Tests (validan que el código cumple el contrato)
```

Los ADR del modelo (ej.: `ADR-003-autenticacion-jwt.md`) muestran cómo se documenta una decisión de seguridad; la consigna del TP4 (`trabajos-practicos/tp4-arquitectura-web/`) define los requisitos del contrato.

> ERROR FRECUENTE
>
> escribir el YAML después de la API, mirando el código para saber qué documentar. El contrato tiene que ir **antes**: es la especificación que la API debe cumplir. Si la API se escribió primero, el contrato está sesgado.

Threat model lite: STRIDE

El entregable `DEL-S3-02` pide un threat model con **≥ 5 amenazas** sobre tus endpoints reales, usando el marco STRIDE.

CONCEPTO CLAVE

STRIDE: acrónimo de 6 categorías de amenazas. Cada letra es una pregunta:

Letra	Amenaza	Pregunta
S	Spoofing (suplantación)	¿Alguien puede hacerse pasar por otro?
T	Tampering (manipulación)	¿Alguien puede modificar datos en tránsito?
R	Repudiation (repudio)	¿Alguien puede negar haber hecho algo?
I	Information Disclosure (fuga)	¿Alguien no autorizado puede leer datos?
D	Denial of Service (denegación)	¿Alguien puede dejar el sistema fuera de servicio?
E	Elevation of Privilege (escalada)	¿Alguien puede hacer cosas de admin sin serlo?

Para cada endpoint de tu API, recorrés las 6 preguntas. Si alguna tiene respuesta afirmativa (o "no lo sé"), es una amenaza que necesita mitigación.

```
Endpoint: POST /solicitudes
  - S: el request no tiene autenticación → riesgo alto
  - T: los datos viajan en HTTP → riesgo medio (pasa a HTTPS)
  - I: el body contiene datos personales → riesgo alto (cifrar en tránsito)
  - E: cualquier usuario puede crear solicitudes → riesgo medio (validar rol)
```

Mitigaciones aplicadas

Cada amenaza identificada tiene que tener una **mitigación aplicada en el código** y documentada en `docs/seguridad/`. No alcanza con listar las amenazas: hay que mostrar qué se hizo.

Amenaza	Mitigación	Dónde se ve
Spoofing	JWT + OAuth2 en todos los endpoints protegidos	`ADR-003`, código de autenticación
Manipulación	HTTPS forzado a nivel de servidor nginx	`docs/seguridad/https.md`
Fuga de datos	Respuestas sin campos sensibles (modelo de datos)	`schemas.py` en dominio

EN EL PROYECTO

En tu sprint (Sprint 3, Fase 1-2): creá `api-contracts.yaml` con 5 endpoints, corré el linter, implementá la API. Después completá el threat model y aplicá las mitigaciones. Todo queda documentado en `docs/seguridad/`.

Secretos: donde no deben estar

Un tema recurrente en las auditorías es la exposición de secretos. Checklist de verificación:

¿Dónde está tu secreto?	¿Puede filtrarse en un commit?
Variables de entorno	No, si `.env` está en `.gitignore`
Archivo `.env` en el repo	**Sí.** No debe estar.
Hardcodeado en el código	**Sí.** No debe estar.
En un vault de CI (GitHub Secrets)	No. Correcto.
En un log	**Sí.** Los logs se suben a veces.

La herramienta `gitleaks` escanea el historial de git en busca de secretos expuestos. Se ejecuta en el pipeline de CI del cierre.

RECORDÁ

Un secreto expuesto no se arregla con un nuevo commit. Si subiste una clave a git, esa clave está comprometida desde ese commit para siempre. Rotá la clave inmediatamente. No confíes en "la borré después".

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué se escribe primero: el `api-contracts.yaml` o el código de la API? ¿Por qué?
2. ¿Qué significa cada letra de STRIDE? (nombrá al menos 3)
3. ¿Dónde nunca debe estar un secreto? ¿Qué hacés si uno quedó expuesto en un commit?

Siguiente unidad: *U9 → Mobile medible y despliegue*

Mobile medible y despliegue funcional

El Sprint 3 cierra con dos entregables que definen si tu proyecto es “una app que funciona” o “una app que se puede mostrar”.

La decisión mobile: ADR-006

Antes de adaptar una pantalla a mobile, tenés que decidir **qué estrategia mobile usás**. Esa decisión se documenta como un ADR (ADR-006) con una **matriz de decisión explícita**: filas = criterios de TU contexto, columnas = opciones, **celdas justificadas** (así la pide la consigna del TP5):

Criterio (TU contexto)	Responsive	PWA	Nativa	Híbrida
¿Costo de mantenimiento con 1 desarrollador?	✅	✅	❌	❌
¿Necesitás uso offline real?	❌	✅	✅	✅
¿Necesitás hardware del dispositivo (cámara, sensores)?	❌	❌	✅	❌
¿Cuánto tiempo queda hasta el cierre de PP3?	✅	✅	❌	~

No hay puntaje: las celdas no se suman. Se justifican. Si tu app no necesita offline ni hardware, responsive/PWA gana la matriz para tu contexto aunque “Nativa” tenga más ✓. La consigna del TP5 lo dice explícito: *premia la decisión fundamentada, no la tecnología cara*.

Presupuestos de rendimiento

La diferencia entre “se ve bien en mobile” y “funciona en mobile” son los **números**. El TP5 pide presupuestos **numéricos** con la **herramienta concreta** que los verifica:

Métrica	Presupuesto	Cómo se mide
LCP (Largest Contentful Paint)	< 2.5 s en 4G	Lighthouse CI
INP (Interaction to Next Paint)	< 200 ms	Lighthouse CI
Peso JS inicial	< 200 KB gzip	`source-map-explorer`

Los presupuestos son para las **2 pantallas críticas del TP3** reinterpretadas en breakpoint móvil (< 400 px), y al menos uno debe poder correr en CI. El resultado se documenta en `docs/arquitectura/presupuestos-rendimiento.md`.

> **CONCEPTO CLAVE**
>
> Presupuesto de rendimiento: un número que decidís **antes** de medir y que no podés superar. Si el presupuesto es LCP < 2.5 s y tu página tarda 4 s, el error no es de la medición: es que no cumpliste el presupuesto y necesitás optimizar.

La herramienta la define la consigna del TP5 (Lighthouse CI, `source-map-explorer`) — no la inventás vos.

Despliegue funcional

El entregable `DEL-S3-04` es `docs/deploy/deploy.md`. No necesitás una URL pública (aunque es mejor si la tenés). Necesitás un documento que describa:

Sección	Pregunta
Stack de despliegue	¿Con qué está montado? (Docker, nginx, servidor VPS, Cloudflare…)
Pasos para reproducir	Si otro desarrollador clona tu repo, ¿qué comandos ejecuta para tenerlo funcionando?
Decisiones de infraestructura	¿Por qué elegiste ese hosting y no otro?
Postmortem	¿Qué salió mal durante el despliegue? ¿Qué harías distinto?

```
docs/deploy/deploy.md
├── Stack: Docker + nginx + Cloudflare Tunnel
├── Pasos:
│   1. docker-compose up --build
│   2. nginx config en ./deploy/nginx.conf
│   3. Tunnel: cloudflared tunnel --url https://tudominio.trycloudflare.com
├── Por qué esto y no otro: …
└── Lo que salió mal: …
```

> **EN EL PROYECTO**
>
> En tu sprint (Sprint 3, Fase 3-4): primero decidí la estrategia mobile (ADR-006), medí presupuestos con la herramienta asignada, adaptá las pantallas críticas a breakpoint móvil (≥ 48 px targets táctiles) y documentá el despliegue. El deploy no tiene que ser perfecto: tiene que estar documentado.

RECORDÁ

“Medido” no es lo mismo que “estimado”. Una captura de Lighthouse corriendo hoy es evidencia. “Creo que anda bien” no lo es. Si no hay captura o salida de herramienta, el presupuesto no está medido.

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué criterios usa la matriz de decisión mobile y cómo justificás la opción elegida?
2. ¿Qué presupuestos define TP5 y con qué herramienta se verifica cada uno?
3. ¿Qué cuatro secciones debe tener `deploy.md` ?

Siguiente unidad: *U10 → Documentación legal y licencias*

Documentación legal y licencias

El programa de PP3 incluye una unidad entera (U6) dedicada a legislación informática. No es un trámite separado del proyecto: es parte del entregable.

Datos personales y responsabilidades

Si tu sistema maneja datos de personas reales (alumnos, ciudadanos, técnicos, clientes), tenés obligaciones legales. No importa que sea un proyecto académico: si los datos existen, hay responsabilidad.

> CONCEPTO CLAVE
>
> **Dato personal:** cualquier información que identifique o pueda identificar a una persona física (nombre, DNI, email, dirección IP, foto). En Argentina, la Ley 25.326 de Protección de Datos Personales regula su tratamiento.

> EN SIMPLE
>
> si en tu sistema alguien se registra con nombre y email, ese es un dato personal. No lo compartas, no lo guardes de más, no lo subas a GitHub aunque sea "solo para pruebas". Usá datos sintéticos (falsos, generados) para desarrollo.

El apéndice avanzado de testing de consistencia (bonus +1) exige **datos sintéticos aislados, repetibles y protegidos**. No se puede usar datos reales de personas en las pruebas.

Propiedad intelectual y licenciamiento

Todo software tiene una licencia que dice qué puede hacer otro con tu código. Elegir la correcta es parte del entregable.

Licencia	Permite uso comercial	Permite modificar	Obliga a compartir mejoras
MIT	Sí	Sí	No
Apache 2.0	Sí	Sí	No (pero pide atribución)
GPL v3	Sí (con condiciones)	Sí	Sí
CC BY-NC	No (solo no comercial)	Sí	Sí (con atribución)

Para proyectos académicos institucionales, las más comunes son MIT o Apache 2.0. El docente puede sugerir una si el proyecto lo justifica.

EN EL PROYECTO

En tu proyecto: agregá un archivo `LICENSE` en la raíz de tu repo con la licencia elegida. En `docs/` o en el README, documentá por qué elegiste esa y no otra. Eso cuenta como parte de "Documentación técnica y legal" (10% de la nota).

Documentación legal mínima

No necesitás un tratado de abogacía. Necesitás que quede claro:

Documento	Qué dice
`LICENSE`	Bajo qué condiciones se puede usar tu código.
README o `docs/legal.md`	Qué datos personales manejás, cómo los protegés, qué licencia elegiste y por qué.
`docs/seguridad/`	Cómo implementaste controles de acceso y cifrado (mitigaciones del threat model).

ERROR FRECUENTE

pensar que "es un proyecto de facultad, no hace falta licencia". Sin licencia, la ley aplica el régimen más restrictivo (todos los derechos reservados). Eso significa que ni el docente puede reutilizar tu código como ejemplo sin pedirte permiso explícito. Ponerle MIT te protege a vos y a la institución.

Ética profesional

La U6 del programa PP3 incluye actitudinales de "ética profesional". En concreto para el proyecto:

- No incluir datos personales reales en commits ni tests.
- No compartir claves, tokens ni accesos ajenos.
- Reconocer qué parte del código generó una IA y qué parte revisaste vos mismo (está en el Anexo I del programa y en la plantilla de PR).
- Defender tus decisiones técnicas con fundamentos, no con "porque lo vi en internet".

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Qué es un dato personal y qué ley lo regula en Argentina?
2. ¿Qué licencia aplicaste a tu repo y qué permite que otros hagan con tu código?
3. ¿Qué datos usás en los tests y por qué no podés usar datos de personas reales?

Siguiente unidad: *U11 → Auditoría, evaluación, bonus y defensa*

Auditoría, evaluación, bonus y defensa

Todo lo que hiciste en los 3 sprints converge acá: la rúbrica que te evalúa, el bonus que te puede sumar un punto, y la defensa donde mostrás todo.

Cómo se evalúa PP3

Los pesos oficiales están en el Programa y el Contrato Pedagógico. Son fijos:

Área	%	¿Qué mira?
Implementación (funcionalidad y calidad de código)	**40%**	Features funcionando, código limpio, cumplimiento de SPEC
Testing y calidad	**20%**	Suite de tests, reportes, cobertura de Non-Goals
Seguridad	**15%**	Threat model, mitigaciones aplicadas, secretos no expuestos
Documentación técnica y legal	**10%**	SPEC, ADR, informes, licencia, deploy.md
Gestión del proyecto	**15%**	Commits, PRs, Kanban, cumplimiento de hitos

```
 40%  Implementación
 20%  Testing y calidad
 15%  Seguridad
 10%  Documentación
 15%  Gestión
─────
100%
```

Las rúbricas deterministas

Cada sprint tiene una rúbrica que se publica al abrir el sprint. No hay sorpresas: los criterios son públicos y se pueden revisar antes de entregar.

La rúbrica del Sprint 1 está en `proyecto-pp3-2026/docs/auditoria/RUBRICA_AUDITORIA_S1.md`. Tiene ítems como:

```
Criterio (rúbrica real de S1)          Peso   Indicador (Score 0-100)          Evidencia
Git Flow y Trazabilidad                25%    ramas feature/*, PRs con         git log y
PRs                                           plantilla, commits trazables
Auditoría Crítica de Código            35%    errores de código asistido       docs/auditoria/
                                              por IA documentados y corregidos
Calidad Automatizada Local             20%    linter/formatter corriendo;      PRs y
configs                                       checklists completos             de lint
Seguridad Base                         20%    cero leaks de secrets,           revisión
                                              .gitignore funcional             estática
```

La rúbrica real no es “todo sí o no”: cada criterio tiene un peso, un indicador medible de 0 a 100 y la evidencia que lo verifica.

> CONCEPTO CLAVE
>
> **Rúbrica determinista:** instrumento de evaluación donde los criterios, los pesos, los indicadores y las evidencias están definidos **previamente** para reducir la arbitrariedad. Puede contener indicadores binarios, puntajes o ponderaciones explícitas — lo determinista es que la regla se conoce antes de entregar, no que todo sea sí/no.

Los informes de auditoría

Cada sprint se cierra con un informe en `docs/auditoria/auditoria-sprintN.md`. Debe contener:

1. **Checklist del sprint completo** con estado de cada ítem.
2. **Errores hallados en código asistido por IA** y correcciones con evidencia (links a PRs).
3. **Estado del sprint** (PASS / FAIL con observaciones).

El informe no lo escribe el docente: **lo escribís vos**. Es parte de la evidencia de que revisaste tu propio trabajo.

> ERROR FRECUENTE
>
> escribir el informe sin registrar el resultado de la revisión. El informe debe documentar qué inspeccionaste, qué evidencia revisaste y el resultado obtenido. Si encontraste un defecto, documentá el hallazgo y su corrección con links a PRs; si no encontraste defectos, documentá qué revisaste y cómo lo verificaste. Los hallazgos ficticios también son un problema: inventar errores es tan malo como no revisar.

Bonus: Apéndice de Testing de Consistencia y Trazabilidad (+1 punto)

Los estudiantes que completen el apéndice avanzado (consigna completa en `proyecto-adi-2026/trabajos-practicos/apendice-avanzado-testing-consistencia/`) pueden sumar hasta **+1 punto** sobre la nota final del cierre integrador.

Criterio	Bonus
Matriz de trazabilidad con fuentes vigentes y oráculo identificado	+0.20
Un flujo crítico con 3 escenarios significativos	+0.25
Datos sintéticos aislados, repetibles y protegidos	+0.20
Comparación ejecutada y resultado documentado con evidencia de desvío o constancia de consistencia	+0.20
Defensa individual satisfactoria	+0.15
Total máximo	**+1.00**

Reglas: - Se aplica solo al cierre integrador de PP3. - El trabajo base debe estar aprobado. - La nota final no puede superar 10. - No sustituye entregables obligatorios. - No se duplica en ADI. - Una suite generada por IA que el estudiante no pueda explicar **no recibe bonus**. - Una prueba sin aserciones significativas no cuenta. - Una suite inestable (flaky) no cumple determinismo.

> EN SIMPLE
> no es “hacer más tests”. Es demostrar que entendiste qué estás testeando, con datos falsos pero realistas, y que podés explicarlo en la defensa.

Las 10 reglas de la defensa

La defensa oral es individual. Dura 10 minutos (7 de presentación + 3 de preguntas). Las preguntas del docente se basan en esta lista:

#	Pregunta que vas a tener que responder
1	**Explicá el flujo elegido y por qué es crítico.**
2	**Identificá el oráculo de cada test.**
3	**Explicá cada escenario y por qué esos tres.**
4	**Distinguí unitario, integración y E2E con ejemplos de tu proyecto.**
5	**Explicá los datos sintéticos y por qué no usás personas reales.**
6	**Mostrá una aserción significativa y explicá qué verifica.**
7	**Explicá un hallazgo o resultado de la inspección humana.**
8	**Distinguí automatización de juicio humano.**
9	**Explicá qué harías ante un secreto expuesto.**
10	**Reconocé qué parte generó la IA y qué validaste personalmente.**

Cada una de estas preguntas está directamente vinculada a una sección de este cuadernillo. Si seguiste las unidades, las respuestas las tenés.

> **RECORDÁ**
>
> **No se evalúa "cuánto código escribiste". Se evalúa "cuánto entendés de lo que hiciste".** Un proyecto chico pero bien defendido puede sacar más nota que uno enorme pero que no se puede explicar.

Cierre: 17 de noviembre

El último hito del año tiene dos fechas distintas:

```
CIERRE TÉCNICO — 17 de noviembre
  Pipeline CI en verde (protegiendo main)
  + release v0.1.0 etiquetada, con CHANGELOG
  + Informe Final PP3 + demo (docs/informe-final.md)

DEFENSA INTEGRADORA (oral) — 19 de noviembre
```

El cierre técnico es el **17 de noviembre** y la defensa integradora es el **19 de noviembre** (fechas del SSOT).

EN EL PROYECTO

En tu cronograma: los PRs del Sprint 3 se mergean antes del cierre del sprint, el **13 de noviembre**. Luego viene el cierre técnico (17 de noviembre: pipeline CI + release v0.1.0 con CHANGELOG + informe final) y la defensa integradora (19 de noviembre).

Checklist de cierre (lo que llevás a la defensa)

- [ ] Release v0.1.0 taggeada en GitHub, con CHANGELOG.
- [ ] Pipeline CI configurado y pasando (al menos lint + tests).
- [ ] 3 informes de auditoría (`auditoria-sprint1/2/3.md`).
- [ ] `docs/deploy/deploy.md` completo.
- [ ] Licencia elegida y documentada.
- [ ] (Si aplica) Apéndice de testing de consistencia.
- [ ] Repositorio público, con README que explica de qué se trata.
- [ ] **Vos podés explicar cada línea de código que dice ser tuya.**

CHECKPOINT — RESPONDÉ SIN MIRAR

1. ¿Cuáles son los 5 pesos de evaluación y su porcentaje?
2. ¿Cuánto suma el bonus del apéndice y qué condición hace que una suite generada por IA no lo reciba?
3. Nombrá tres de las diez reglas de la defensa.
4. ¿Qué tres entregables formales se rinden en el cierre técnico del 17 de noviembre? ¿Cuándo es la defensa integradora?

Fin del cuadernillo PP3 — Coordinación del Proyecto Integrador.

Encontraste un error? Reportalo en `github.com/IES9018/proyecto-pp3-2026/issues`.